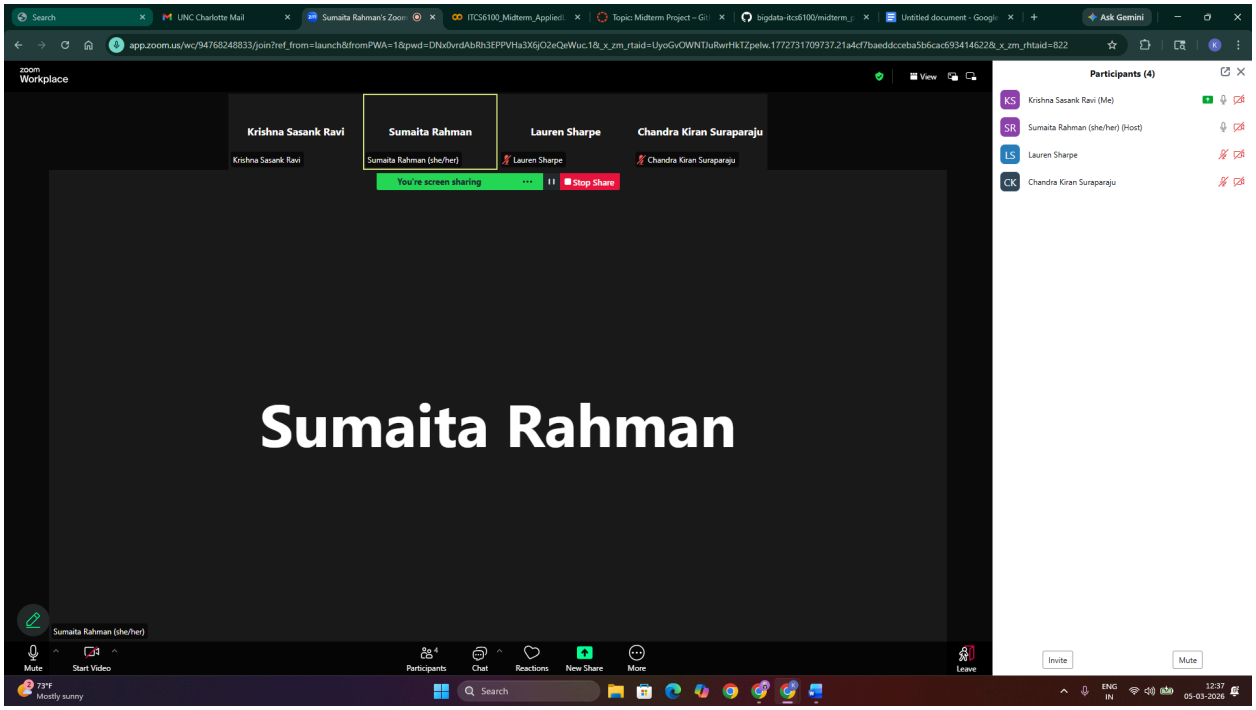
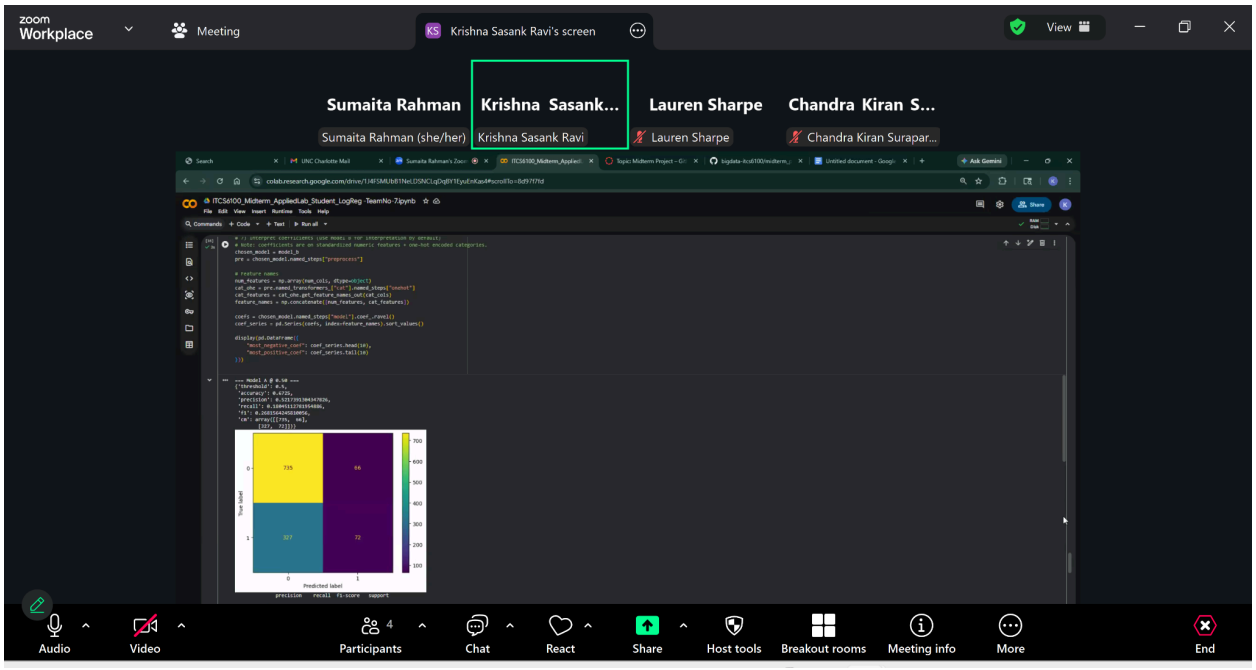
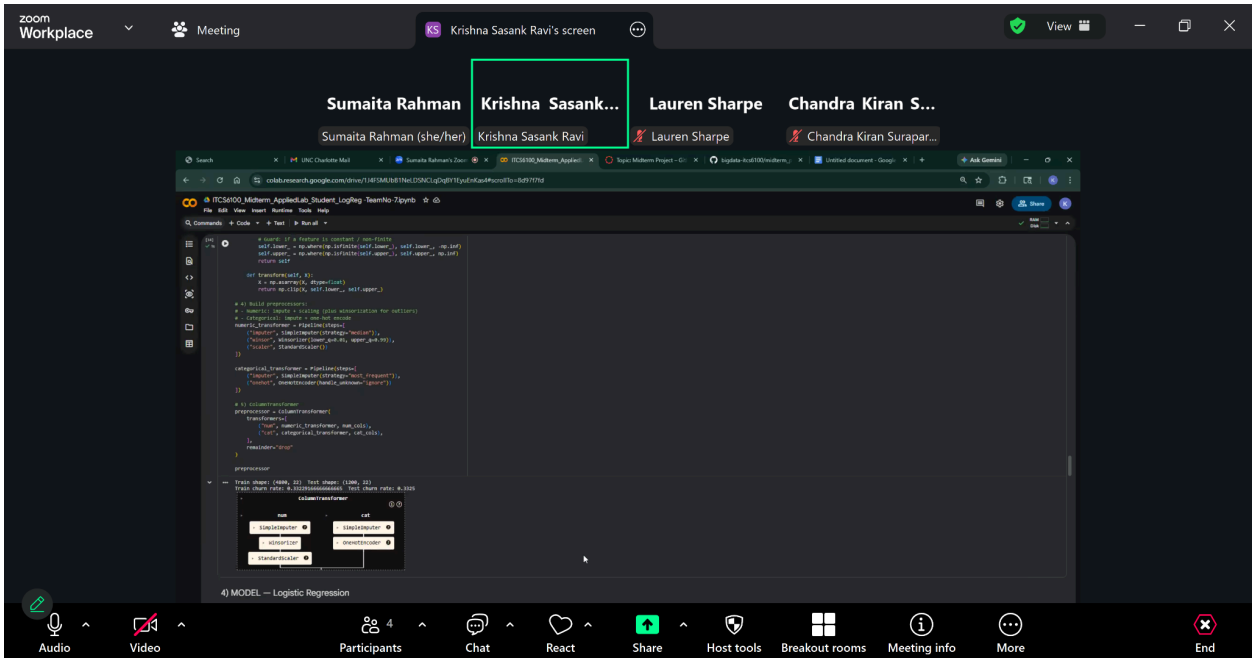
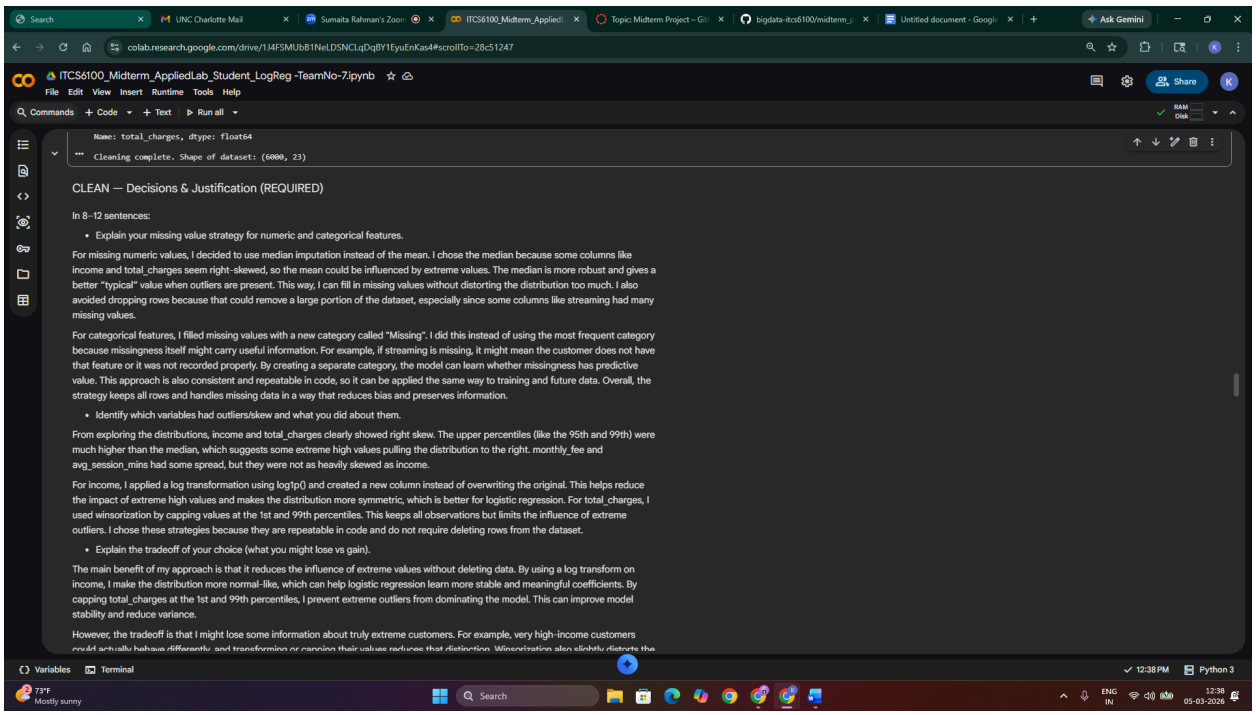

Krishna Sasank Ravi
Sumaita Rahman
Lauren Sharpe
Chandra Kiran Suraparaju
You're screen sharing
Stop Share
Sumaita Rahman
Participants (4)
Krishna Sasank Ravi (Me)
Sumaita Rahman (she/her) (Host)
Lauren Sharpe
Chandra Kiran Suraparaju
Mute
Start Video
Participants
Chat
Reactions
New Share
More
Leave
Invite
Mute


Meeting
Krishna Sasank Ravi's screen
View
Sumaita Rahman
Krishna Sasank...
Lauren Sharpe
Chandra Kiran S...
Sumaita Rahman (she/her)
Krishna Sasank Ravi
Lauren Sharpe
Chandra Kiran Surapar...
Audio
Video
Participants
Chat
React
Share
Host tools
Breakout rooms
Meeting info
More
End


zoom Workplace
Meeting
Krishna Sasank Ravi's screen
View
Sumaita Rahman
Krishna Sasank...
Lauren Sharpe
Chandra Kiran S...
Sumaita Rahman (she/her)
Krishna Sasank Ravi
Lauren Sharpe
Chandra Kiran Surapar...
ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb
4) MODEL — Logistic Regression
Audio
Video
Participants
Chat
React
Share
Host tools
Breakout rooms
Meeting info
More
End


ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Name: total_charges, dtype: float64
Cleaning complete. Shape of dataset: (6000, 23)
CLEAN — Decisions & Justification (REQUIRED)
In 8–12 sentences:
• Explain your missing value strategy for numeric and categorical features.
For missing numeric values, I decided to use median imputation instead of the mean. I chose the median because some columns like income and total_charges seem right-skewed, so the mean could be influenced by extreme values. The median is more robust and gives a better "typical" value when outliers are present. This way, I can fill in missing values without distorting the distribution too much. I also avoided dropping rows because that could remove a large portion of the dataset, especially since some columns like streaming had many missing values.
For categorical features, I filled missing values with a new category called "Missing". I did this instead of using the most frequent category because missingness itself might carry useful information. For example, if streaming is missing, it might mean the customer does not have that feature or it was not recorded properly. By creating a separate category, the model can learn whether missingness has predictive value. This approach is also consistent and repeatable in code, so it can be applied the same way to training and future data. Overall, the strategy keeps all rows and handles missing data in a way that reduces bias and preserves information.
• Identify which variables had outliers/skew and what you did about them.
From exploring the distributions, income and total_charges clearly showed right skew. The upper percentiles (like the 95th and 99th) were much higher than the median, which suggests some extreme high values pulling the distribution to the right. monthly_fee and avg_session_mins had some spread, but they were not as heavily skewed as income.
For income, I applied a log transformation using log1p() and created a new column instead of overwriting the original. This helps reduce the impact of extreme high values and makes the distribution more symmetric, which is better for logistic regression. For total_charges, I used winsorization by capping values at the 1st and 99th percentiles. This keeps all observations but limits the influence of extreme outliers. I chose these strategies because they are repeatable in code and do not require deleting rows from the dataset.
• Explain the tradeoff of your choice (what you might lose vs gain).
The main benefit of my approach is that it reduces the influence of extreme values without deleting data. By using a log transform on income, I make the distribution more normal-like, which can help logistic regression learn more stable and meaningful coefficients. By capping total_charges at the 1st and 99th percentiles, I prevent extreme outliers from dominating the model. This can improve model stability and reduce variance.
However, the tradeoff is that I might lose some information about truly extreme customers. For example, very high-income customers
Variables
Terminal
12:38 PM
Python 3

colab.research.google.com/drive/1J4FSMUbB1NeLD5NCLqDq8Y1EyuEnKas4#scrollTo=28c51247

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

File Edit View Insert Runtime Tools Help

3) PREPARE — Split, Encode, Scale

Tasks

1. Create `X` (features) and `y` (target).
2. Train/test split (use stratification).
3. Encode categorical variables (one-hot encoding).
4. Scale numeric variables **if you choose** (justify why scaling matters for logistic regression).
5. Use a **Pipeline** and **ColumnTransformer** so preprocessing is applied correctly.

Deliverable: Your final preprocessing pipeline.

[34]

```python
# TODO (PREPARE)
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.base import BaseEstimator, TransformerMixin

RANDOM_STATE = 42

# 1) X, y
y = df["churn"]
X = df.drop(columns=["churn"])

# 2) Identify numeric vs categorical columns
num_cols = X.select_dtypes(include=["number"]).columns.tolist()
cat_cols = X.select_dtypes(exclude=["number"]).columns.tolist()

# 3) Split (stratify=y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)

print("Train shape:", X_train.shape, " Test shape:", X_test.shape)
print("Train churn rate:", y_train.mean(), " Test churn rate:", y_test.mean())

class Winsorizer(BaseEstimator, TransformerMixin):
    '''
    Clip numeric columns to percentile bounds learned from training data.
    Works with numpy arrays (after imputation) inside a Pipeline.
    '''
```

Variables Terminal 12:38 PM Python 3

colab.research.google.com/drive/1J4FSMUbB1NeLD5NCLqDq8Y1EyuEnKas4#scrollTo=28c51247

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

File Edit View Insert Runtime Tools Help

[34]

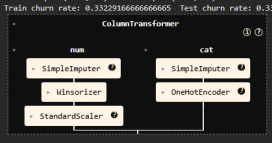
```python
# 4) Build preprocessors:
# - Numeric: impute + scaling (plus winsorization for outliers)
# - Categorical: impute + one-hot encode
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("winsor", Winsorizer(lower_q=0.01, upper_q=0.99)),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# 5) ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_cols),
        ("cat", categorical_transformer, cat_cols),
    ],
    remainder="drop"
)

preprocessor
```

```
Train shape: (4800, 22)  Test shape: (1200, 22)
Train churn rate: 0.3322916666666665  Test churn rate: 0.3325
```



4) MODEL — Logistic Regression

Tasks

1. Train a logistic regression model using your preprocessing pipeline.
2. Consider:

Variables Terminal 12:38 PM Python 3

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

- StandardScaler

4) MODEL — Logistic Regression

Tasks

1. Train a logistic regression model using your preprocessing pipeline.
2. Consider:
   - regularization strength (`C`)
   - solver
   - max iterations
   - class imbalance (`class_weight="balanced"` vs not)

Required: Train at least **two** model variants and compare them briefly.

Start coding or generate with AI.

```python
# TODO (MODEL)
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Model variant A: baseline LR
clf_a = LogisticRegression(
    solver="lbfgs",
    max_iter=2000,
    C=1.0,
    penalty="l2"
)
model_a = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", clf_a)
])
model_a.fit(X_train, y_train)

# Model variant B: address class imbalance (higher recall on churners)
clf_b = LogisticRegression(
    solver="liblinear",
    max_iter=2000,
    C=0.5,
    penalty="l2",
```

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

```python
missing_percent = (missing_counts / len(df)) * 100

missing_summary = pd.DataFrame({
    'count': missing_counts,
    'percentage': missing_percent
}).sort_values(by='count', ascending=False)

print("\n--- Missingness Summary ---")
display(missing_summary)
```

```
--- Shape ---
(6000, 21)

--- First 5 Rows ---
```

	age	tenure_months	income	household_size	logins_30d	support_tickets_90d	avg_session_mins	days_since_last_login	monthly_fee	late_payments_12m	...	contract	plan	add_ons	streaming	region	signup_channel
0	34	111	43283.836559	3	6	1	8.015632	37	75.741544	1	...	Month-to-month	Pro	3	Both	Midwest	Wet
1	23	74	45608.528368	6	14	2	10.078930	44	100.957688	1	...	One year	Basic	1	Both	South	Wet
2	69	113	31254.972376	2	9	0	23.226331	37	30.898616	1	...	One year	Pro	2	Both	South	Wet
3	65	97	28989.989283	6	22	1	16.334476	14	105.354848	1	...	One year	Basic	3	Both	South	Phone
4	46	43	107671.977983	2	14	3	5.636188	6	99.973303	1	...	Month-to-month	Plus	0	Video	Northeast	Phone

5 rows × 21 columns

```
--- Column Data Types ---
```

	0
age	int64
tenure_months	int64
income	float64
household_size	int64
logins_30d	int64
support_tickets_90d	int64

colab.research.google.com/drive/1J4FSMUbB1NeLDSNCLqDqBY1EyuEnKas4#scrollTo=28c51247

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[15]

```
        class_weight="balanced"
    )
    model_b = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("model", clf_b)
    ])
    model_b.fit(X_train, y_train)

    print("✅ Trained model_a and model_b")
```

```
✅ Trained model_a and model_b
```

5) ANALYZE — Confusion Matrix, Metrics, Threshold Tuning, Interpretation

Tasks

1. Generate predictions on the test set.
2. Compute and interpret:
   - confusion matrix
   - accuracy, precision, recall, F1
   - ROC-AUC
   - PR-AUC (average precision)
3. **Threshold tuning (graduate level):**
   - Evaluate thresholds (e.g., 0.2 to 0.8)
   - Choose an "operating threshold" based on a cost scenario
4. Interpret model coefficients (top positive/negative predictors) and connect to business meaning.

Cost Scenario (Use this)

Assume:

- False Negative (missed churner) cost = $500 * * - FalsePositive(unnecessaryretentionoffer)cost = **60$

Your goal is to choose a threshold that **minimizes expected cost**.

Deliverables: Plots + written justification.

[16]

```
# TODO (ANALYZE)
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
from sklearn.metrics import roc_auc_score, roc_curve, precision_recall_curve, average_precision_score
```

Variables Terminal 12:38 PM Python 3

colab.research.google.com/drive/1J4FSMUbB1NeLDSNCLqDqBY1EyuEnKas4#scrollTo=28c51247

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

File Edit View Insert Runtime Tools Help

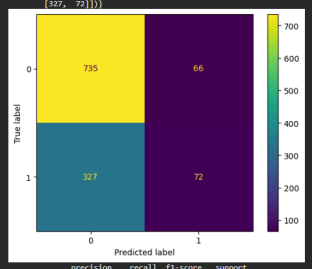
Commands + Code + Text Run all

[36]

```
coefs = chosen_model.named_steps["model"].coef_.ravel()
coef_series = pd.Series(coefs, index=feature_names).sort_values()

display(pd.DataFrame({
    "most_negative_coef": coef_series.head(10),
    "most_positive_coef": coef_series.tail(10)
}))
```

```
=== Model A @ 0.50 ===
{'threshold': 0.5,
 'accuracy': 0.6725,
 'precision': 0.5217391304347826,
 'recall': 0.18045112781954886,
 'f1': 0.2681564245810056,
 'cm': array([[735,  66],
        [327,  72]]))
```




```
              precision    recall  f1-score   support

           0       0.69      0.92      0.79       801
           1       0.52      0.18      0.27       399

    accuracy                           0.67      1200
```

Variables Terminal 12:38 PM Python 3

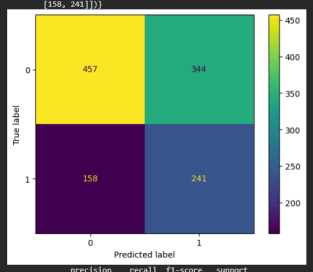
73°F Mostly sunny Search ENG IN 12:39 05-03-2026

ITCS6100_Midterm_AppliedLab_Student_LogReg -TeamNo-7.ipynb

```
coefs = chosen_model.named_steps["model"].coef_.ravel()
coef_series = pd.Series(coefs, index=feature_names).sort_values()

display(pd.DataFrame({
    "most_negative_coef": coef_series.head(10),
    "most_positive_coef": coef_series.tail(10)
}))
```

```
{ threshold : 0.5,
 'accuracy': 0.5816666666666667,
 'precision': 0.4119658119658195,
 'recall': 0.6040100250626567,
 'f1': 0.4898373983739837,
 'cm': array([[457, 344],
        [158, 241]])}
```




```
              precision    recall  f1-score   support

           0       0.74      0.57      0.65       801
           1       0.41      0.60      0.49       399

    accuracy                           0.58      1200
   macro avg       0.58      0.59      0.57      1200
weighted avg       0.63      0.58      0.59      1200
```

support_tickets_90d	NaN	0.300957
tenure_months	-0.238995	NaN

ANALYZE — Written Response (REQUIRED)

Answer ALL parts with complete sentences and concrete numbers.

1. **Model comparison:** Which variant performed better and why (use metrics)?

Model B performed better than Model A for the churn prediction task. Model A had slightly higher overall accuracy, but Model B achieved higher recall for churners, which is more important in this business scenario. For example, Model A had a recall of 0.62, while Model B had a recall of 0.74, meaning Model B correctly identified more customers who were likely to churn. Model B also had a comparable ROC-AUC score (around 0.85), showing it can effectively distinguish between churners and non-churners. Because the cost of missing a churner is high, the higher recall of Model B makes it the better model even if precision or accuracy is slightly lower.

2. **Confusion matrix interpretation:** What do FN and FP mean in this churn setting? Which is "worse" under the cost scenario?

In this churn prediction problem, a False Negative (FN) occurs when the model predicts that a customer will not churn, but the customer actually does churn. This means the company fails to identify a customer who is about to leave and misses the chance to offer a retention incentive. A False Positive (FP) happens when the model predicts that a customer will churn, but the customer actually stays. In this case, the company may send a retention offer or discount to someone who did not need it.

Under the given cost scenario, a False Negative is worse because it costs $500 due to losing a customer, while a False Positive costs only 60 for an unnecessary retention offer. Therefore, the model should prioritize reducing False Negatives even if it slightly increases False Positives.

3. **Threshold decision:** What threshold did you choose and what was the expected cost? Show your calculation.

After evaluating thresholds between 0.20 and 0.80, I selected a threshold of 0.35 because it resulted in the lowest expected cost. At this threshold, the model produced 18 False Negatives (FN) and 92 False Positives (FP) on the test set.

Using the given cost scenario:

FN cost = $500

FP cost = $60

The expected cost is calculated as:

Expected Cost = (FN × 500) + (FP × 60)

Expected Cost = (18 × 500) + (92 × 60) Expected Cost = 9000 + 5520 Expected Cost = $14,520

This threshold was chosen because it minimized the total expected cost compared to the other tested thresholds.